Programare în C pentru platforma Linux (I)

Gestiunea fișierelor, partea I-a: Primitivele I/O pentru lucrul cu fișiere

Cristian Vidrașcu
vidrascu@info.uaic.ro

Martie, 2020

Sumar

Sumar

Introducere

Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere

Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare

Biblioteca standard de C: funcții pentru operații I/O cu fișiere

Referințe bibliografice

- Introducere – principalele categorii de primitive I/O
- Principalele apeluri de sistem POSIX pentru operații I/O
- Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare
- Funcțiile I/O din biblioteca standard de C
- Referințe bibliografice

Sumar

Introducere

Principalele categorii de primitive I/O (1/2)

Principalele categorii de primitive I/O (2/2)

Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere

Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare

Biblioteca standard de C: funcții pentru operații I/O cu fișiere

Referințe bibliografice

Introducere

Principalele categorii de primitive I/O (1/2)

Sumar

Introducere
Principalele categorii de primitive I/O (1/2)
Principalele categorii de primitive I/O (2/2)

Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere

Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare

Biblioteca standard de C: funcții pentru operații I/O cu fișiere

Referințe bibliografice

Sistemul de gestiune a fișierelor în `UNIX`/`Linux` furnizează următoarele categorii de apeluri sistem în standardul POSIX:

- primitive de creare de noi fișiere, de diverse tipuri: `mknod`, `mkfifo`, `mkdir`, `link`, `symlink`, `creat`, `socket`
- primitive de ștergere a unor fișiere: `rmdir` (pentru directoare), `unlink` (pentru toate celelalte tipuri)
- primitiva de redenumire a unui fișier, de orice tip: `rename`
- primitive de consultare a *i*-nodului unui fișier: `stat`/`fstat`/`lstat`, `access`
- primitive de manipulare a *i*-nodului unui fișier: `chmod`/`fchmod`, `chown`/`fchown`/`lchown`,
- primitive de extindere a sistemului de fișiere: `mount`, `umount`
- primitive de accesare și manipulare a conținutului unui fișier, printr-o “sesiune de lucru”: `open`/`creat`, `read`, `write`, `lseek`, `close`, `fcntl`
- primitive de duplicare a unei “sesiuni de lucru” cu un fișier: `dup`, `dup2`

Principalele categorii de primitive I/O (2/2)

Sumar

Introducere

Principalele categorii de primitive I/O (1/2)

Principalele categorii de primitive I/O (2/2)

Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere

Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare

Biblioteca standard de C: funcții pentru operații I/O cu fișiere

Referințe bibliografice

- primitive pentru consultarea “stării” unor “sesiuni de lucru” cu fișiere (operații I/O sincrone multiplexate): `select`, `poll`
- primitive de modificare a unor atribute dintr-un proces: `chdir` (directorul curent de lucru), `umask` (masca permisiunilor implicite la crearea unui fișier), `chroot` (rădăcina sistemului de fișiere accesibil procesului)
- primitive pentru acces exclusiv la fișiere: `flock`, `fcntl`
- primitiva de “mapare” a unui fișier în memoria unui proces: `mmap`
- primitiva de creare, într-un proces, a unui canal de comunicație anonim: `pipe`
- ș.a.

Observație: în caz de eroare toate aceste primitive returnează valoarea `-1`, precum și un număr de eroare ce este stocat în variabila globală `errno` (definită în fișierul *header* `errno.h`).

Sumar

Introducere

Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere

Primitiva access

Primitiva creat

Primitiva open

Primitiva read

Primitiva write

Primitiva lseek

Primitiva close

Exemplu de sesiune de lucru

Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare

Biblioteca standard de C: funcții pentru operații I/O cu fișiere

Referințe bibliografice

Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere

Primitiva access

Sumar
Introducere
Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere
Primitiva access
Primitiva creat
Primitiva open
Primitiva read
Primitiva write
Primitiva lseek
Primitiva close
Exemplu de sesiune de lucru
Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare
Biblioteca standard de C: funcții pentru operații I/O cu fișiere
Referințe bibliografice

- *Verificarea drepturilor de acces la un fișier*: primitiva `access`.

  Interfața funcției `access`:

  `int access(char* nume_cale, int drept)`

  - *nume_cale* = numele fișierului
  - *drept* = dreptul de acces ce se verifică, ce poate fi o combinație (*i.e.*, disjuncție logică pe biți) a următoarelor constante simbolice:
    - `X_OK` (=1) : procesul apelant are drept de execuție a fișierului ?
    - `W_OK` (=2) : procesul apelant are drept de scriere a fișierului ?
    - `R_OK` (=4) : procesul apelant are drept de citire a fișierului ?

    Notă: pentru *drept*=`F_OK` (=0) se verifică doar existența fișierului.
  - valoarea returnată este `0`, dacă accesul(ele) verificat(e) este/sunt permis(e), respectiv `-1` în caz de eroare.

Primitiva creat

Sumar
Introducere
Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere
Primitiva access
Primitiva creat
Primitiva open
Primitiva read
Primitiva write
Primitiva lseek
Primitiva close
Exemplu de sesiune de lucru
Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare
Biblioteca standard de C: funcții pentru operații I/O cu fișiere
Referințe bibliografice

- *Crearea de fișiere de tip obișnuit*: primitiva `creat`.

  Interfața funcției `creat`:

  `int creat(char*` *nume_cale*`, int` *perm_acces*`)`

  — *nume_cale* = numele fișierului ce se creează
  — *perm_acces* = drepturile de acces pentru noul fișier creat
  — valoarea returnată este descriptorul de fișier deschis, sau `-1` în caz de eroare.

  Efect: în urma execuției funcției `creat` se creează fișierul specificat și este "deschis" în scriere (!), valoarea returnată având aceeași semnificație ca la `open`.

  Observație: în cazul când acel fișier deja există, el este trunchiat la zero, păstrându-i-se drepturile de acces pe care le avea.

  Notă: practic, un apel `creat(`*nume_cale*`,` *perm_acces*`);` este echivalent cu apelul următor:

  `open(`*nume_cale*`, O_WRONLY|O_CREAT|O_TRUNC,` *perm_acces*`);`

Primitiva open

Sumar
Introducere
Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere
Primitiva access
Primitiva creat
Primitiva open
Primitiva read
Primitiva write
Primitiva lseek
Primitiva close
Exemplu de sesiune de lucru
Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare
Biblioteca standard de C: funcții pentru operații I/O cu fișiere
Referințe bibliografice

- *"Deschiderea" unui fișier*, *i.e.* inițializarea unei "sesiuni de lucru": primitiva `open`.
  Interfața funcției `open`:
  `int open(char*` *nume_cale*`, int` *tip_desch*`, int` *perm_acces*`)`
  - *nume_cale* = numele fișierului ce se deschide
  - *perm_acces* = drepturile de acces pentru fișier (utilizat numai în cazul în care apelul va avea ca efect crearea acelui fișier)
  - *tip_desch* = specifică tipul deschiderii, putând fi exact una singură dintre valorile `O_RDONLY` ori `O_WRONLY` ori `O_RDWR`, și, eventual, combinată cu o combinație (*i.e.*, disjuncție logică pe biți) a următoarelor constante simbolice: `O_APPEND`, `O_CREAT`, `O_TRUNC`, `O_EXCL`, `O_CLOEXEC`, `O_NONBLOCK`, ș.a.
  - valoarea returnată este descriptorul de fișier deschis (*i.e.*, **indexul în tabela locală de fișiere deschise**), sau `-1` în caz de eroare.

Primitiva read

Sumar
Introducere
Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere
Primitiva access
Primitiva creat
Primitiva open
Primitiva read
Primitiva write
Primitiva lseek
Primitiva close
Exemplu de sesiune de lucru
Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare
Biblioteca standard de C: funcții pentru operații I/O cu fișiere
Referințe bibliografice

- *Citirea dintr-un fișier*: primitiva `read`.

  Interfața funcției `read`:

  `int read(int` *df*`, char*` *buffer*`, unsigned` *nr_oct*`)`

  — *df* = descriptorul fișierului din care se citește
  — *buffer* = adresa de memorie la care se depun octeții citiți
  — *nr_oct* = numărul de octeți de citit din fișier
  — valoarea returnată este numărul de octeți efectiv citiți, dacă citirea a reușit (chiar și parțial), sau `-1` în caz de eroare.

Observații: i) numărul de octeți efectiv citiți poate fi inclusiv 0, dacă la începutul citirii cursorul în fișier este pe poziția `EOF`(*i.e. end-of-file*);
ii) la sfârșitul citirii cursorul va fi poziționat pe următorul octet după ultimul octet efectiv citit.

Primitiva write

Sumar
Introducere
Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere
Primitiva access
Primitiva creat
Primitiva open
Primitiva read
Primitiva write
Primitiva lseek
Primitiva close
Exemplu de sesiune de lucru
Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare
Biblioteca standard de C: funcții pentru operații I/O cu fișiere
Referințe bibliografice

- *Scrierea într-un fișier*: primitiva `write`.

  Interfața funcției `write`:

  `int write(int` *df*`, char*` *buffer*`, unsigned` *nr_oct*`)`

  — *df* = descriptorul fișierului în care se scrie
  — *buffer* = adresa de memorie al cărei conținut se scrie în fișier
  — *nr_oct* = numărul de octeți de scris în fișier
  — valoarea returnată este numărul de octeți efectiv scriși, dacă scrierea a reușit (chiar și parțial), sau `-1` în caz de eroare.

  Observație: la sfârșitul scrierii cursorul va fi poziționat pe următorul octet după ultimul octet efectiv scris.

Primitiva lseek

Sumar
Introducere
Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere
Primitiva access
Primitiva creat
Primitiva open
Primitiva read
Primitiva write
Primitiva lseek
Primitiva close
Exemplu de sesiune de lucru
Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare
Biblioteca standard de C: funcții pentru operații I/O cu fișiere
Referințe bibliografice

- *Poziționarea cursorului într-un fișier* (*i.e.* ajustarea deplasamentului curent în fișier): primitiva `lseek`.

  Interfața funcției `lseek`:

  `long lseek(int` *df*`, long` *val_ajust*`, int` *mod_ajust*`)`

  - *df* = descriptorul fișierului ce se poziționează
  - *val_ajust* = valoarea de ajustare a deplasamentului
  - *mod_ajust* = modul de ajustare, indicat după cum urmează:
    - `SEEK_SET` (=0) : ajustare în raport cu începutul fișierului
    - `SEEK_CUR` (=1) : ajustare în raport cu deplasamentul curent
    - `SEEK_END` (=2) : ajustare în raport cu sfârșitul fișierului
  - valoarea returnată este noul deplasament în fișier (întotdeauna, în raport cu începutul fișierului), sau `-1` în caz de eroare.

Primitiva close

Sumar
Introducere
Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere
Primitiva access
Primitiva creat
Primitiva open
Primitiva read
Primitiva write
Primitiva lseek
Primitiva close
Exemplu de sesiune de lucru
Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare
Biblioteca standard de C: funcții pentru operații I/O cu fișiere
Referințe bibliografice

- *"Închiderea" unui fișier*, *i.e.* finalizarea unei "sesiuni de lucru": primitiva `close`.
  Interfața funcției `close`:
  `int close(int df)`
  - *df* = descriptorul de fișier deschis
  - valoarea returnată este `0`, dacă închiderea a reușit, respectiv `-1` în caz de eroare.

Primitiva close

Sumar
Introducere
Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere
Primitiva access
Primitiva creat
Primitiva open
Primitiva read
Primitiva write
Primitiva lseek
Primitiva close
Exemplu de sesiune de lucru
Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare
Biblioteca standard de C: funcții pentru operații I/O cu fișiere
Referințe bibliografice

- *"Închiderea" unui fișier*, *i.e.* finalizarea unei "sesiuni de lucru": primitiva `close`.
  Interfața funcției `close`:
  `int close(int df)`
  - *df* = descriptorul de fișier deschis
  - valoarea returnată este `0`, dacă închiderea a reușit, respectiv `-1` în caz de eroare.

Observație: O "sesiune de lucru", *i.e.* maniera uzuală de prelucrare a unui fișier, constă în următoarele: "deschiderea fișierului", urmată de o buclă de parcurgere a acestuia cu operații de citire și/sau de scriere, și eventual cu schimbări ale poziției curente în fișier, iar în final "închiderea" acestuia.

Exemplu: a se vedea programele filtru dos2unix.c și unix2dos.c.

Exemplu de sesiune de lucru

Sumar

Introducere

Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere

Primitiva access

Primitiva creat

Primitiva open

Primitiva read

Primitiva write

Primitiva lseek

Primitiva close

Exemplu de sesiune de lucru

Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare

Biblioteca standard de C: funcții pentru operații I/O cu fișiere

Referințe bibliografice

Iată un exemplu de program ce realizează o “sesiune de lucru” cu fișiere, mai exact realizează o copiere secvențială a unui fișier dat:

```
/* Basic cp file copy program. POSIX implementation. */
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>
#include <errno.h>
#define BUF_SIZE 512  /* or 4096 */

int main (int argc, char *argv []) {
    int input_fd, output_fd;
    ssize_t bytes_in, bytes_out;
    char buffer[BUF_SIZE];
    if (argc != 3) {
        printf("Usage: cp file1 file2\n");  return 1;
    }
    input_fd = open(argv [1], O_RDONLY);
    if (input_fd == -1) {
        perror(argv [1]);  return 2;
    }
    output_fd = open(argv[2], O_WRONLY|O_CREAT|O_TRUNC, 0600);
    if (output_fd == -1) {
        perror(argv [2]);  return 3;
    }

    /* Process the input file a record at atime. */
    while ((bytes_in = read(input_fd, &buffer, BUF_SIZE)) > 0) {
        bytes_out = write(output_fd, &buffer, bytes_in);
        if (bytes_out != bytes_in) {
            perror("Fatal write error.");  return 4;
        }
    }
    close(input_fd);   close(output_fd);
    return 0;
}
```

Sumar

Introducere

Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere

Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare

Alte primitive I/O pentru fișiere (1/2)

Alte primitive I/O pentru fișiere (2/2)

Primitive I/O pentru directoare

Șablonul de lucru cu directoare

Biblioteca standard de C: funcții pentru operații I/O cu fișiere

Referințe bibliografice

Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare

Alte primitive I/O pentru fișiere (1/2)

Sumar
Introducere
Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere
Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare
Alte primitive I/O pentru fișiere (1/2)
Alte primitive I/O pentru fișiere (2/2)
Primitive I/O pentru directoare
Șablonul de lucru cu directoare
Biblioteca standard de C: funcții pentru operații I/O cu fișiere
Referințe bibliografice

- *"Duplicarea" unui descriptor de fișier*: primitivele `dup` și `dup2`.
- *Controlul operațiilor I/O*: primitivele `fcntl` și `ioctl`.
- *Obținerea de informații conținute de i-nodul unui fișier*: primitivele `stat`, `lstat` sau `fstat`.
- *Crearea/ștergerea unei legături pentru un fișier*: primitiva `link`, respectiv `unlink`.
- *Schimbarea drepturilor de acces la un fișier*: primitiva `chmod`.
- *Schimbarea proprietarului unui fișier*: primitivele `chown` și `chgrp`.
- *Configurarea măștii drepturilor de acces la crearea unui fișier*: primitiva `umask`.
- *Montarea/demontarea unui sistem de fișiere*: primitiva `mount`, respectiv `umount`.
- *Crearea pipe-urilor* (*i.e.* canale de comunicație anonime): primitiva `pipe`.
- ș.a.

Alte primitive I/O pentru fișiere (2/2)

Sumar

Introducere

Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere

Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare

Alte primitive I/O pentru fișiere (1/2)

Alte primitive I/O pentru fișiere (2/2)

Primitive I/O pentru directoare

Șablonul de lucru cu directoare

Biblioteca standard de C: funcții pentru operații I/O cu fișiere

Referințe bibliografice

- *Crearea fișierelor de tip fifo* (*i.e.* canale de comunicație cu nume): primitiva `mkfifo`.
  Interfața funcției `mkfifo`:
  `int mkfifo(char* nume_cale, int perm_acces);`
  - *nume_cale* = numele fișierului *fifo* ce se creează
  - *perm_acces* = drepturile de acces pentru acesta
  - valoarea returnată este 0 în caz de succes, sau –1 în caz de eroare.
- *Crearea fișierelor de tip director*: primitiva `mkdir`.
  Interfața funcției `mkdir`:
  `int mkdir(char* nume_cale, int perm_acces);`
  - *nume_cale* = numele directorului ce se creează
  - *perm_acces* = drepturile de acces pentru acesta
  - valoarea returnată este 0 în caz de succes, sau –1 în caz de eroare.
- *Schimbarea directorului curent de lucru, al unui proces*: primitiva `chdir`.
  Interfața funcției `chdir`:
  `int chdir(char* nume_cale);`
  - *nume_cale* = numele noului director curent de lucru, al procesului apelant
  - valoarea returnată este 0 în caz de succes, sau –1 în caz de eroare.
- ș.a.

Primitive I/O pentru directoare

Sumar
Introducere
Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere
Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare
Alte primitive I/O pentru fișiere (1/2)
Alte primitive I/O pentru fișiere (2/2)
Primitive I/O pentru directoare
Șablonul de lucru cu directoare
Biblioteca standard de C: funcții pentru operații I/O cu fișiere
Referințe bibliografice

- *Crearea/ștergerea unui director*: primitiva `mkdir`, respectiv `rmdir`.
- *Aflarea directorului curent de lucru*: primitiva `getcwd`.
- *Schimbarea directorului curent*: primitiva `chdir`.
- *"Prelucrarea" fișierelor dintr-un director*: primitivele `opendir`, `readdir` și `closedir`.

O "sesiune de lucru" cu directoare decurge asemănător ca una cu fișiere, *i.e.* este tot o buclă de forma: "deschidere", operații de citire, "închidere".
Se folosesc structurile de date `DIR` și `struct dirent` și funcțiile enumerate mai sus, în felul următor . . .

Șablonul de lucru cu directoare

Sumar
Introducere
Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere
Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare
Alte primitive I/O pentru fișiere (1/2)
Alte primitive I/O pentru fișiere (2/2)
Primitive I/O pentru directoare
Șablonul de lucru cu directoare
Biblioteca standard de C: funcții pentru operații I/O cu fișiere
Referințe bibliografice

```
DIR           *dd; // descriptor de director deschis
struct dirent *de; // intrare in director

/* deschiderea directorului */

if( (dd = opendir(nume_director)) == NULL)
{
   ...  // trateaza eroarea
}

/* prelucrarea secventiala a tuturor intrarilor din director */

while( (de = readdir(dd)) != NULL)
{
   ...  // prelucreaza intrarea curenta, ce are numele: de->d_name
}

/* inchiderea directorului */

closedir(dd);
```

Sumar

Introducere

Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere

Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare

Biblioteca standard de C: funcții pentru operații I/O cu fișiere

Biblioteca standard de C
Funcțiile I/O din biblioteca standard de C

Exemplu de sesiune de lucru

Referințe bibliografice

Biblioteca standard de C: funcții pentru operații I/O cu fișiere

Biblioteca standard de C

Sumar
Introducere
Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere
Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare
Biblioteca standard de C: funcții pentru operații I/O cu fișiere
Biblioteca standard de C
Funcțiile I/O din biblioteca standard de C
Exemplu de sesiune de lucru
Referințe bibliografice

- C library contains functions with limited capability to manage OS resources (*e.g.*, files)
- Often adequate for simple programs
- Possible to write portable programs, between various platforms (*e.g.*, `Windows`, `UNIX`, ș.a.)
- Include files: <stdlib.h>, <stdio.h>, <string.h>
- Competitive performance
- Still constrained to synchronous I/O
- No control of file security via C library

- Call to fopen() specifies whether file is text or binary
- Open files identified by pointers to FILE structures
  - NULL indicates invalid value
  - Pointers are „handles“ to open file objects
- Errors are diagnosed with perror() or ferror()

Funcțiile I/O din biblioteca standard de C

Sumar
Introducere
Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere
Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare
Biblioteca standard de C: funcții pentru operații I/O cu fișiere
Biblioteca standard de C
Funcțiile I/O din biblioteca standard de C
Exemplu de sesiune de lucru
Referințe bibliografice

Biblioteca standard de C conține un set de funcții I/O (cele din *header*-ul `stdio.h`), care permit și ele prelucrarea unui fișier în maniera uzuală:

- `fopen` = pentru "deschiderea" fișierului
- `fread`, `fwrite` = pentru citire, respectiv scriere binară
- `fscanf`, `fprintf` = pentru citire, respectiv scriere formatată
- `fclose` = pentru "închiderea" fișierului

Observație: acestea sunt funcții de bibliotecă (nu sunt apeluri sistem) și lucrează *buffer*-izat, cu *stream*-uri I/O, iar descriptorii de fișiere utilizați de ele nu sunt de tip `int`, ci de tip `FILE*`.

Notă: implementările acestor funcții de bibliotecă utilizează totuși apelurile de sistem corespunzătoare fiecărei platforme în parte (*i.e.*, `Windows` vs. `UNIX`).

Observație: sunt mult mai multe funcții I/O în biblioteca `stdio.h` ; pentru a vedea lista lor și descrierea bibliotecii standard de I/O, inclusiv detalii despre cele 3 fluxuri I/O standard (*i.e.*, `stdin`, `stdout` și `stderr`), vă recomand consultarea paginii de manual `man 3 stdio` .

Exemplu de sesiune de lucru

Sumar

Introducere

Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere

Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare

Biblioteca standard de C: funcții pentru operații I/O cu fișiere

Biblioteca standard de C Funcțiile I/O din biblioteca standard de C

Exemplu de sesiune de lucru

Referințe bibliografice

Iată un exemplu de program ce realizează o “sesiune de lucru” cu fișiere, mai exact realizează o copiere secvențială a unui fișier dat:

```
/* Basic cp file copy program. C library implementation. */
#include <stdio.h>
#include <errno.h>
#define BUF_SIZE 512  /* or 4096 */

int main (int argc, char *argv []) {
    FILE *input_file, *output_file;
    ssize_t bytes_in, bytes_out;
    char buffer[BUF_SIZE];
    if (argc != 3) {
        printf("Usage: cp file1 file2\n");  return 1;
    }
    input_file = fopen(argv [1], "rb");
    if (input_file == NULL) {
        perror(argv [1]);  return 2;
    }
    output_file = fopen(argv[2], "wb");
    if (output_file == -1) {
        perror(argv [2]);  return 3;
    }

    /* Process the input file a record at atime. */
    while ((bytes_in = fread(buffer, 1, BUF_SIZE, input_file)) > 0) {
        bytes_out = fwrite(buffer, 1, bytes_in, output_file);
        if (bytes_out != bytes_in) {
            perror("Fatal write error.");  return 4;
        }
    }
    fclose(input_file);
    fclose(output_file);
    return 0;
}
```

Sumar

Introducere

Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere

Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare

Biblioteca standard de C: funcții pentru operații I/O cu fișiere

Referințe bibliografice

Bibliografie obligatorie

Referințe bibliografice

Bibliografie obligatorie

Sumar
Introducere
Principalele apeluri de sistem POSIX pentru operații I/O cu fișiere
Alte apeluri de sistem POSIX pentru operații I/O cu fișiere și directoare
Biblioteca standard de C: funcții pentru operații I/O cu fișiere
Referințe bibliografice
Bibliografie obligatorie

Cap.3, §3.1 din manualul, în format PDF, accesibil din pagina disciplinei “Sisteme de operare”:

- `https://profs.info.uaic.ro/~vidrascu/SO/books/ManualID-SO.pdf`

Programele demonstrative amintite pe parcursul acestei prezentări pot fi descărcate de la adresa următoare:

- `https://profs.info.uaic.ro/~vidrascu/SO/cursuri/C-programs/file/`

Întrebări?